

ЛАБОРАТОРНА РОБОТА № 5

Тема: Автоматизація (CI/CD) частина 2 Jenkins.

Мета: Отримати практичний досвід роботи щодо автоматизації рутинних задач.

Час виконання: 2 год.

ХІД РОБОТИ

Теоретичні відомості

Сьогодні створюється і виробляється безліч різної організаційної техніки та гаджетів, які не можуть, і не працюватимуть правильно без належного програмного забезпечення.

Програмісти отримують різні завдання з написання софту, та іноді, під час написання програми, допускаються помилки, через які софт просто не запуститься, або видасть помилку. Її виправлення може зайняти багато часу, що вкрай не вигідно.

Для спрощення та прискорення цього завдання ще у 2008 році було створено Jenkins.

Jenkins – система з відкритим вихідним кодом, тобто продукт доступний для перегляду, вивчення та зміни. Він створений на базі Java.

Надає сотні плагінів для підтримки створення, розгортання та автоматизації будь-якого проекту

Дженкінс дозволяє автоматизувати частину процесу розробки програмного забезпечення без участі людини. Ця система призначена для забезпечення процесу безперервної інтеграції програмного забезпечення.

Переваги Jenkins

Безперервна інтеграція та безперервна доставка

Як розширений сервер автоматизації, Jenkins можна використовувати як простий сервер CI або перетворити на центр безперервної доставки для будь-якого проекту.

Легкий монтаж

Jenkins — це автономна програма на основі Java, готова до запуску «з коробки» з пакетами для Windows, Linux, macOS та інших операційних систем, подібних Unix.

Легке налаштування

Jenkins можна легко налаштувати за допомогою веб-інтерфейсу, який включає перевірку помилок на льоту та вбудовану довідку.

Плагіни

Завдяки сотням плагінів у Центрі оновлення, Jenkins інтегрується практично з кожним інструментом у ланцюзі інструментів безперервної інтеграції та безперервної доставки.

Розширення

Jenkins можна розширити за допомогою архітектури плагінів, надаючи майже нескінченні можливості для роботи.

Поширення

Jenkins може легко розподіляти роботу між кількома машинами, допомагаючи швидше створювати, тестувати та розгортати на кількох платформах. Зараз Jenkins використовується практично в будь-якій сучасній компанії, де є необхідність в автоматичному деплоїменті (розгортанні) додатків, а також у зручному управлінні різноманітних завдань.

CI/CD автоматизація за допомогою Jenkins

Розберемо роль такого продукту, як Jenkins та його аналогів. Програмне забезпечення Jenkins написано мовою програмування Java, за відгуками ІТ спільноти, цей продукт написаний дуже добре. Але найголовніше це програмне забезпечення повністю безкоштовне. Багато ентузіастів у світі для цього продукту пишуть плагіни, які розширюють функціонал Jenkins.

Розглянемо 2 ключові поняття CI/CD Автоматизації.

CI – Continuous Integration. Це DevOps модель, в якій розробники роблять commit коду в репозиторій (зазвичай використовується github або gitlab, для зберігання коду) і автоматично запускається build або компіляція цього коду, після цього запускаються автоматичні тести коду: Unit Test, Integration Test, Functionality Test.

CD – Continuous Delivery and Deployment. Це DevOps модель, в якій розробники роблять commit коду в репозиторій і автоматично запускається build або компіляція цього коду, після цього запускаються автоматичні тести коду і готовий Artifact (компільований код, наприклад, якщо це Java, то артефактом є war, якщо це Android додаток, то apk (файл) робить деплой в Staging і Production, тобто відбувається встановлення коду в розгорнуте ваше середовище в необхідному контурі. Розглянемо процес на прикладі.

Процес CI/CD автоматизації

Першим кроком у процесі є Commit to Source Control (github, gitlab або bitbucket): система визначає наявність нового коду, спрацьовує тригер і автоматично запускається наступний етап Build \ Compile - компіляція коду.

Система завантажує новий код, наприклад, якщо код потрапив у master branch (основну гілку). Після отримання відповіді від складання, що все пройшло успішно, розпочинається наступний етап тестів. Всі тести пишуть ті самі програмісти, для того, щоб перевірити наскільки коректно відпрацював код. Весь цей процес називається

Continuous Integration. Ця класична схема містить три етапи, іноді включаються додаткові кроки, але вони не принципові. В результаті цього процесу ми отримуємо скомпільований та протестований код.

Наступний крок – ми можемо зробити deployment коду. По суті, це той же процес копіювання файлів коду на сервера. Процес деплою можна робити в різні місця, можна робити в AWS або Azure, можна робити в свою приватну хмару, розгорнуту на VMware. Весь процес із додатковими кроками називається Continuous Delivery and Deployment.

Виходить наступне: за Source Control відповідає git. За крок build та compile буде відповідати Jenkins. Отже, Jenkins запуститься, коли хтось зробить коміт у систему контролю версій, в основну гілку чи не основну, дивлячись як налаштовано. Наступним кроком Jenkins виконає всі потрібні тести, які підготували програмісти. Наступний крок Deploy також запустить Jenkins і скопіює код на необхідні сервери, за допомогою скрипта або scp, якщо це Linux сервер. Існують варіації з використанням Puppet або Ansible, якщо ми робимо Deploy артефакту або конфігурації в цілому.

Існують альтернативи Jenkins, наприклад, Bamboo, Circleci, Gitlab CI/CD, TeamCity.

Встановлення Jenkins

Для розгортання Jenkins нам знадобиться віртуальна машина наприклад Ubuntu

Ідемо на офіційний сайт Jenkins, у розділі Download ми можемо побачити 2 версії. На момент написання лабораторної актуальна версія Jenkins 2.361.1 LTS, а у другій колонці ми можемо побачити тижневі версії 2.371

Download Jenkins 2.361.1 LTS for:

Generic Java package (.war) SHA-256: 08a72b43d570f785796a7f8b398d2d4865d20cdd985e524bc33b7f9cd5907eb3
Docker
Ubuntu/Debian
CentOS/Fedora/Red Hat
Windows
openSUSE
FreeBSD 
Gentoo 
macOS 
OpenBSD 

Download Jenkins 2.371 for:

Generic Java package (.war) SHA-256: ecb0610c40a1a7dd24939bfcc63022552e04b72dde64683bf57a1e90e3306522
Docker
Ubuntu/Debian
CentOS/Fedora/Red Hat
Windows
openSUSE
Arch Linux 
FreeBSD 
Gentoo 
macOS 
OpenBSD 
OpenIndiana Hipster 

Як бачите дистрибутиви є практично під усі операційні системи. Ми будемо використовувати стабільну версію під Ubuntu\Debian. Ознайомимось з вимогами до встановлення продукту Jenkins. Для встановлення потрібно мінімум 256 МБ RAM, 1 ГБ місця, а також на сайті написані рекомендовані вимоги, з якими буде досить комфортно працювати з продуктом.

Оскільки Jenkins написаний на Java, то для запуску та роботи буде потрібно безпосередньо встановлена на сервері Java. Спочатку перевіримо версію java на сервері.

```
java -version
```

Якщо Java не встановлено, операційна система повідомить, що така команда не знайдена і запропонувати встановити Java. Java встановлюється досить просто:

sudo apt update – оновлюємо репозиторій

sudo apt search openjdk – шукаємо необхідний пакет

sudo apt install openjdk-11-jdk – запускаємо установку java у процесі система попросить підтвердити. Щоб попередження не вискочило, ми можемо запустити установку з ключем **-y**

Після закінчення установки ми знову перевіряємо версію. Система покаже версію та білд Java. Тепер наш сервер готовий до початку встановлення Jenkins.

Додаємо ключ та репозиторій в операційну систему:

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
```

sudo apt-get update – оновлюємо репозиторій

sudo apt-get install Jenkins – встановлюємо безпосередньо сам Jenkins

Далі продовжується налаштування безпосередньо в Jenkins.

Відкриваємо браузер і переходимо на веб-інтерфейс <http://ipaddr:8080>, де замість ipaddr – підставляємо IP адресу сервера. У відповідь отримуємо таке повідомлення - Unlock Jenkins

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

Система просить ввести додатковий ключ, згенерований під час встановлення сервера. Знайти його досить просто, достатньо ввести в консолі сервера

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Копіюємо та вставляємо у веб форму. Після проходження цієї нескладної системи безпеки ми можемо розпочати базове налаштування.

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

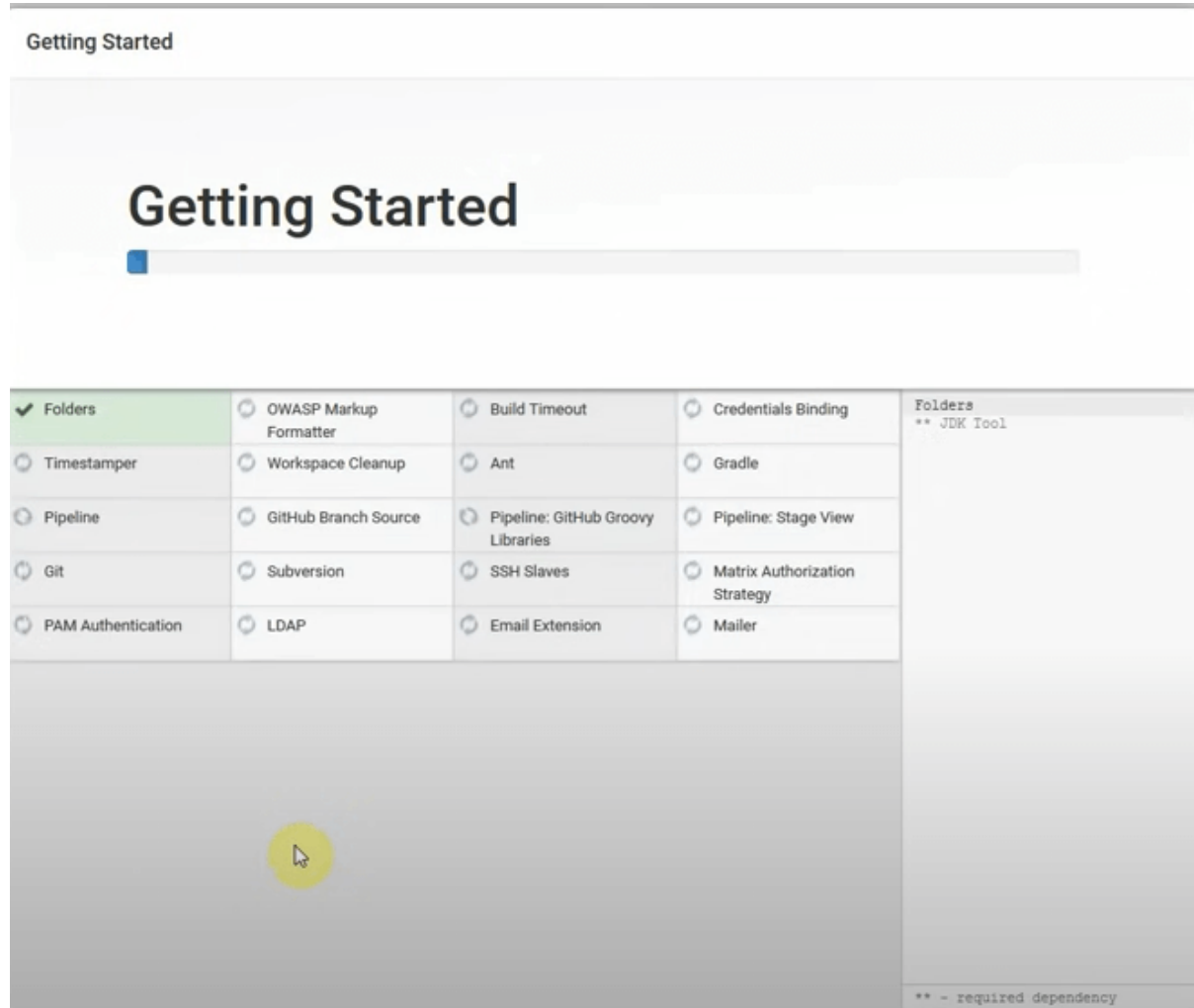
Select plugins to install

Select and install plugins most suitable for your needs.



Система пропонує вибрати стандартну установку або кастомізовану із вибором плагінів (розширень для різного функціоналу). Якщо ми вибираємо стандартну установку, встановлюються лише ті плагіни, які самі розробники протестували та обрали. Якщо ми виберемо установку з вибором, система дасть можливість встановити не тільки стандартні, але й інші плагіни.

Вибираємо стандартну установку і починається процес налаштування самого Jenkins.



Ми бачимо, що ставиться git плагін, LDAP для роботи з Active Directory, ssh для взаємодії протоколу ssh, розширення E-mail для відправлення повідомлень і т.д.

Після не тривалого очікування система пропонує створити суперкористувача з правами адміністратора в системі.

Getting Started

Create First Admin User

Username:

Password:

Confirm password:

Full name:

E-mail address:

Якби ми вибрали інший варіант установки, система нам запропонувала б вибрати самостійно потрібні плагіни.

Getting Started

Organization and Administration

Build Features

Build Tools

Build Analysis and Reporting

Pipelines and Continuous Delivery

Source Code Management

Distributed Builds

User Management and Security

Notifications and Publishing

All | None | Suggested

Selected (20/57)

☐ Rebuilder ↗

This plugin is for rebuilding a job using the same parameters.

10 ↵

☐ SSH Agent ↗

This plugin allows you to provide SSH credentials to builds via a ssh-agent in Jenkins

8 ↵

☐ Throttle Concurrent Builds ↗

This plugin allows for throttling the number of concurrent builds of a project running per node or globally.

13 ↵

☒ Timestamper ↗

Adds timestamps to the Console Output

4 ↵

☒ Workspace Cleanup ↗

This plugin deletes the project workspace when invoked.

14 ↵

Build Tools (2/4)

☒ Ant ↗

Adds Apache Ant support to Jenkins

4 ↵

☒ Gradle ↗

This plugin allows Jenkins to invoke [Gradle](#) build scripts directly.

5 ↵

☐ MSBuild ↗

This plugin makes it possible to build a Visual Studio project (.proj) and solution files (.sln).

5 ↵

☐ NodeJS ↗

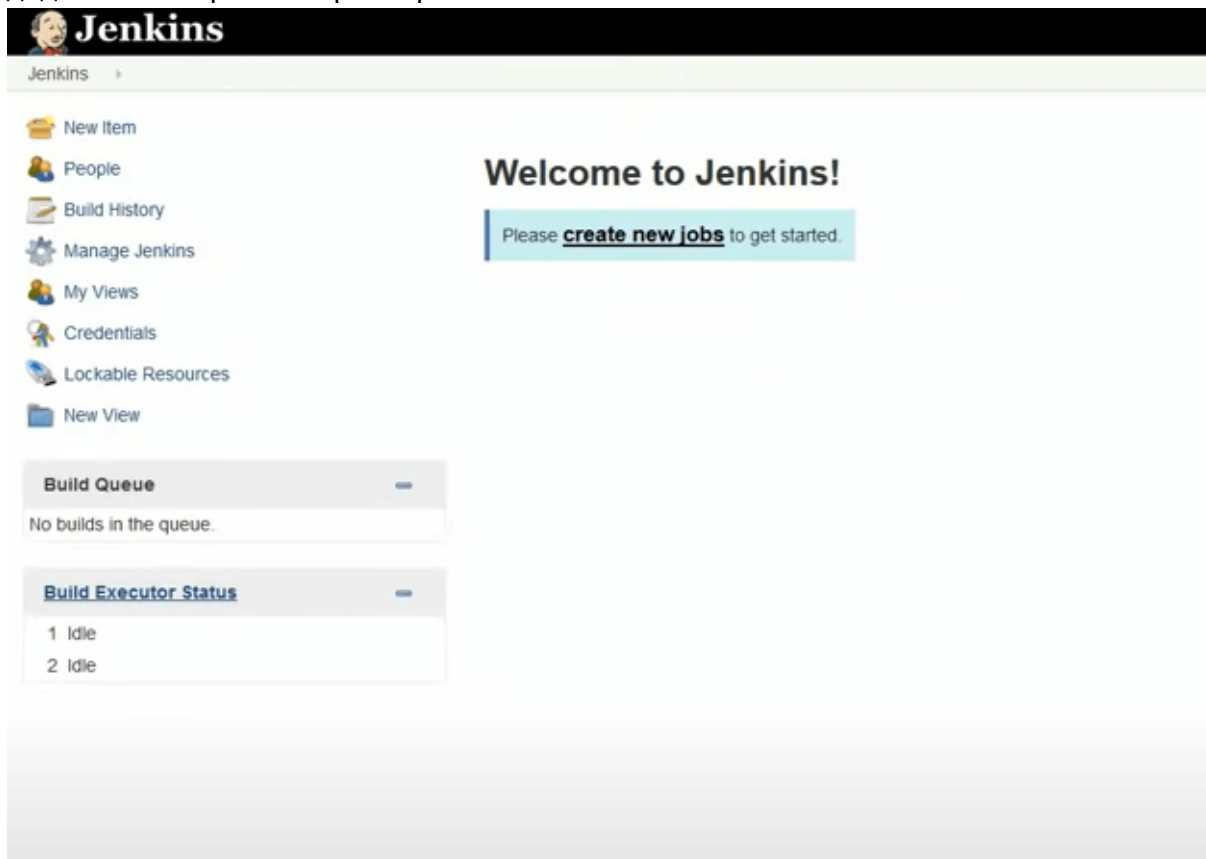
NodeJS Plugin executes [NodeJS](#) script as a build step.

10 ↵

Build Analysis and Reporting (0/6)

Після завершення заповнення форми, потрапляємо на екран де нам пропонують перевірити URL, тому що ці дані буде використовувати Jenkins, як змінні середовища.

Зрештою ми потрапляємо на головний екран Jenkins. Цей екран – це основний робочий стіл. За допомогою плагінів його можна кастомізувати. Також можна в джоби додати багато різних параметрів.



Завдання для виконання:

1. Встановити Jenkins
2. Зробити джобу в Jenkins, що буде запускати ansible playbook з попередньої лабораторної роботи
3. Оформити звіт про виконання роботи зі знімками екрана (або його частини), які ілюструють виконання завдань

Підказки, як це зробити

<https://medium.com/appgambit/ansible-playbook-with-jenkins-pipeline-2846d4442a31>

<https://devopsbuzz.com/run-ansible-playbook-from-jenkins/>

<https://habr.com/ru/company/southbridge/blog/585776/>